

4 • Prototype Design

This chapter documents the design and construction of the RECS toolkit — the research instrument this thesis contributes. It describes the problem the prototype solves, the design decisions taken along the way, the conceptual shift that separated a promising-but-incorrect first build from the final v2 instrument, and the resulting architecture. A companion rulebook (Appendix B) covers operation; this chapter is concerned with rationale and process.

4.1 Starting point

The toolkit takes as its conceptual anchor Sturdee, Heron & Gamboa's (2023) CHI Extended Abstract *TTRPG UX: Requirements & Beyond*, which proposes a framework called the Requirements Elicitation Campaign Setting (RECS) — a method by which a UX researcher plays the role of a tabletop role-playing game's gamesmaster (GM), the participant plays a persona character, and a dice-driven mechanic introduces narrative complications during a user-research session. Sturdee et al. argue that the psychological distance of role-play allows a participant to articulate usability needs that traditional observational methods miss, and that the dice mechanic operationalises the "edge cases" often absent from standard usability tests.

The paper stops at the level of a framework sketch: a persona character sheet schema (their Figure 3), a description of the Blades-in-the-Dark-inspired dice mechanic with a fivefold outcome band, and a call for the method to be instantiated. It does not publish a running implementation. Our task was to take RECS from a proposal to a researcher-usable tool — one that a small UX team could pick up, run a session with, and produce a consistent data artefact from.

At the outset we had a second constraint. The thesis evaluates the toolkit empirically using Poco, a Figma-prototyped ADHD productivity assistant that one of the authors had already built. However, the toolkit's value — and the contribution we wanted to make — is as a *general* instrument for UX research, not a Poco-specific artefact. This distinction shaped every subsequent decision: Poco became the **test vehicle** for validating the toolkit, not the domain of the toolkit itself.

4.2 Design goals

From the paper and the project constraints we extracted six goals for the instrument.

1. **Faithful to the proposed framework.** Every core mechanic in Sturdee et al. — the persona character sheet, the d20-plus-modifier roll, the difficulty-class banding, the complication draw — should be present and operational. Departures from the paper must be justified.
2. **Product-agnostic core, product-specific packs.** The toolkit must be usable by any UX researcher for any product. Scenarios specific to a product should live in swappable modules.
3. **Session-complete in one sitting.** A researcher should be able to launch the toolkit, run a session, and walk away with a single exportable record, with no setup beyond opening a browser page.
4. **Researcher-facing, not participant-facing.** The tool is an instrument the GM drives. The participant's locus of attention is the real product on their own device; the toolkit's screen is secondary.

5. **Low barrier to fork and extend.** Future researchers should be able to author a new scenario pack without a build step, toolchain, or framework lock-in.
6. **Producing a data artefact that supports thematic analysis.** The export must contain, for each session: the persona sheet, every beat encountered, every roll and its outcome, every injected complication, live GM notes, participant quotes with timestamps, and post-session debrief — all in a form that a coder can read directly.

4.3 Platform choice: HTML/JSX, not Figma

An early decision was whether to build the toolkit as an interactive Figma file or as a browser-runnable web artefact. The original proposal assumed Figma; we overturned that assumption early.

Figma offered the advantage of fitting a design-student workflow and making the toolkit shareable as a single link. But it carried three costs. First, Figma's prototyping mode cannot hold mutable state — rolling a d20 requires a random value that persists, attribute allocation requires constrained arithmetic, and exporting a session record requires generating structured text. None of these are possible without injecting JavaScript via plugins or workarounds that break the "open the link and run" promise. Second, the artefact's GM-facing logic — dice, complication library, export formatting — would need to be reconstructed every time a researcher copied the file. Third, a Figma-based toolkit cannot be run headless for automated testing, which limited our ability to verify correctness before the pilot.

A browser-runnable HTML/JSX prototype solves all three. We chose a zero-build-step architecture: a single `index.html` loads React 18 and Babel Standalone from a CDN and mounts JSX source files via `<script type="text/babel" src="...">` tags. Data persistence uses the browser's `localStorage`. A researcher clones the folder, opens `index.html` in any modern browser, and is ready to run a session. A forked pack is a single `.jsx` file added to the `packs/` folder with its script tag.

This approach trades Figma's design-tool affordances for three gains: real state, automated testability, and a file-per-concern source layout that supports research collaborators modifying one piece without understanding the whole. The visual fidelity of the toolkit is therefore not carried by Figma but by a shared stylesheet (`colors_and_type.css`) that already existed in the Poco design-system repository, plus a toolkit-specific `toolkit.css`. The aesthetic decision — parchment panels, rune-style typography for the GM areas, and a clean Poco-palette for the participant areas — reinforces the dual character of the tool: half research-instrument, half role-play artefact.

4.4 Architecture

The toolkit is organised as eleven JSX modules plus a registry for scenario packs.

GameData.jsx	– core rules: attributes, dice, outcomes, complication library
Packs.jsx	– pack registry API (register / get / list / scenario)
packs/<name>.jsx	– one file per scenario pack, self-registers on load
packs/_schema.md	– authoring reference for pack authors
Store.jsx	– session state + localStorage persistence
Primitives.jsx	– shared UI primitives (Btn, Field, Modal, Nav, SectionTitle, ...)
Dice.jsx	– the dice-roll modal with animation and outcome classification
SetupScreen.jsx	– stage 1: participant code + pre-session checklist
CharacterScreen.jsx	– stage 2: persona character sheet (Session Zero)
ScenarioPickerScreen.jsx	– stage 3: pack picker + scenario picker + pre-read
ScenarioScreen.jsx	– stage 4: beat-by-beat runner
GMNotebook.jsx	– side panel visible during stages 3 and 4
SummaryScreen.jsx	– stage 5: stats, debrief textarea, Markdown + JSON export
App.jsx	– root shell, stage router, rules modal

The session is a single mutable object held in React state and mirrored to `localStorage` on every change. The object contains the participant code, the chosen pack and scenario, the persona character, the index of the current beat, an array of completed beats, arrays of rolls and complications, GM notes, timestamped participant quotes, session timestamps, and the post-session debrief text. One session per browser tab; starting a new session overwrites it (with a confirmation prompt) unless exported first.

The pack registry is a pure namespace. Each pack file calls `window.RECS.Packs.register({ id, name, scenarios, ... })` at load time. The pack picker reads the registered list; the scenario runner reads the active pack's scenarios by id. Adding a pack thus requires only writing a JSX file and adding one `<script>` tag. We ship two reference packs: `poco` (three Poco-specific scenarios covering morning launch, focused work under interruption, and cognitive overwhelm) and `bill-payment` (a single generic scenario — "pay the electricity bill" — intended as a minimal demonstration that the toolkit works for any consumer product).

4.5 Translation from Sturdee et al. to running code

To keep the prototype honest to its source, we maintained an explicit mapping between concepts in the paper and artefacts in the code. The table below records that mapping.

PAPER CONCEPT	RECS-TOOLKIT REALISATION	LOCATION IN CODE
Persona Character Sheet (Fig. 3)	<code>CharacterScreen.jsx</code> with portrait, name/role/age, UXP, 6 attributes with steppers, device/app/trait tag selectors, backstory	<code>CharacterScreen.jsx</code>
Attributes (UX capacities)	6 generic attributes adapted from D&D's 6-stat structure: PAT, FOC, WMEM, EREG, TLIT, SENS, each with gloss and example	<code>GameData.jsx</code>
Difficulty Class (DC)	Three bands — Easy 10, Medium 14, Hard 18 — attached per challenge beat by the pack author	<code>GameData.jsx</code> , per-beat in packs
d20 mechanic with fivefold banding	<code>classifyRoll(d20, total, dc)</code> returns <code>crit / ok / mix / fail / crit-fail</code> ; dice modal presents the roll with animation	<code>GameData.jsx</code> , <code>Dice.jsx</code>
UXP (User Experience Points)	Session-state field on the character; re-roll for 1, cancel complication for 2; +1 on a natural 20	<code>Store.jsx</code> , <code>ScenarioScreen.jsx</code>
Stress clock (Blades-in-the-Dark inspired)	0–6 counter incremented on each injected perturbation; at 6, character trauma-exits	<code>Store.jsx</code> , <code>CharacterScreen.jsx</code>
Complication library	18 core complications (4 categories × 3 severity tiers), plus per-pack extras merged at draw time	<code>GameData.jsx</code> , per-pack <code>complicationsExtra</code>
GM role	GM Notebook side panel with hints per beat, live notes textarea, quote capture, roll + complication log	<code>GMNotebook.jsx</code>
Session Zero	Character screen explicitly labelled "Session Zero — Persona Character Sheet"; sheet-ready gate (name + full 18-point allocation) prevents advancing without a real character	<code>CharacterScreen.jsx</code>
Session data artefact	Markdown export (human-readable session record) and JSON export (full structured dump)	<code>SummaryScreen.jsx</code> → <code>buildMarkdown()</code>

Departures from the paper are explicitly noted below in §4.6 and §4.7.

4.6 Design pivot: perturbation injection

The single most consequential design decision was a correction made during v1 of the prototype, after which the toolkit was re-architected into v2.

In v1 we built the dice mechanic as a *success judgement*. The d20 roll was framed as "did the persona succeed at this task?" — a natural reading for anyone familiar with D&D, where a Strength check determines whether a character breaks down a door. This framing produced a coherent-looking prototype, but in doing so it replicated the problem that RECS is designed to escape. A traditional usability test *also* judges whether the user succeeded; adding a die to that judgement does not elicit anything new. Worse, it pulled the participant's

attention away from the real product and onto the toolkit: the participant would wait to see what the die said, then react to the die rather than to their actual experience.

Re-reading Sturdee et al. §4.1 made the intended reading clear. The paper borrows not D&D's success/failure but Blades in the Dark's *success with complication* — a mechanic in which the question is not "did it work?" but "what happens around it?". The dice do not decide whether the participant's action on the real product succeeded. They decide **what the environment injects** at that moment: calm, a mild complication, a significant complication, or a compound one. The participant's own behaviour on the real product is the outcome; the die is only the mechanism by which edge-case conditions are introduced into the session without requiring the researcher to improvise them.

This reframe changed the toolkit in four ways.

1. **Outcome vocabulary.** The outcome labels were rewritten from success/failure language (*"pass"*, *"fail"*, *"critical success"*) to environmental language (*"calm"*, *"mild perturbation"*, *"significant perturbation"*, *"compound perturbation"*).
2. **Narration overrides.** The pack beat schema gained `onCalm`, `onMild`, and `onSignificant` fields — pack-author-supplied narration for each outcome tier. The scenario runner now reads these aloud verbatim when an outcome lands. Previous tier names (`onSuccess`, `onFailure`) were semantically wrong and were retired.
3. **GM Notebook labels.** The GM-facing hints for a challenge beat were rewritten from "Call for a check" with implicit success framing to "Call for a check; the roll decides what the environment does, not whether they succeed." The log panel was re-titled *Roll & perturbation log*.
4. **Challenge beat content.** Each challenge beat now carries a `taskPrompt` separate from its `setup` narration. The `setup` establishes fiction; the `taskPrompt` names the concrete real-product action the participant is attempting. This separation matters because the dice do not adjudicate the `taskPrompt` — they layer perturbation over the moment the participant is in the middle of it.

The v1-to-v2 pivot removed roughly 800 lines of code and replaced them with approximately the same volume of differently-shaped code. The functional surface of the tool did not shrink. What changed was its semantic centre.

4.7 Other departures from the paper

Beyond the perturbation-injection reframe, three smaller departures are worth noting.

No condition selector in the toolkit. A study comparing TTRPG-style elicitation to traditional usability testing needs a *condition* for each session. The paper does not mandate the implementation of this field in the instrument itself. Our v1 prototype included a condition selector on the setup screen ("traditional" vs "TTRPG"), which routed the session to either a simplified stopwatch-plus-notes view or the full RECS runner. We removed this in v2 on the grounds that (a) a TTRPG UX toolkit should not carry its comparison condition as a feature — that belongs in the study protocol — and (b) a researcher comparing conditions will use the toolkit only for the TTRPG arm; the traditional arm belongs in separate infrastructure (screen recording plus observation template). Study design and instrument are separated accordingly.

Point budget of 18 across six attributes. Sturdee et al. suggest attribute allocation but do not specify a budget. We chose 18, which is the sum that leaves every attribute at 3 (neutral) by default; any reallocation is thus a *deliberate* deviation from neutrality, which reads as persona-building rather than optimisation. This is a judgement call, subject to revision after the pilot.

Starting UXP of 4 and stress cap of 6. These too are not specified in the paper. We chose 4 UXP and a 6-point stress clock (matching Blades in the Dark's stress range) as conservative starting points. Pilot data may suggest adjustments; the values are single constants in `GameData.jsx` and can be tuned without restructuring.

4.8 Session lifecycle

A session proceeds through five stages, gated by data requirements:

1. **Setup** — participant code required (and nothing else).
2. **Character** — persona name required, and the 18 attribute points must be fully allocated, for the "Sheet ready →" button to activate.
3. **Picker** — a pack and a scenario within it must both be selected.
4. **Run** — the scenario is walked beat-by-beat. Narration beats advance with a Continue button; challenge beats open a dice modal that walks the roll, shows the narration override, presents any drawn complication, and offers UXP spends before advancing. Participant quotes and GM notes are captured in the side panel as the session runs.
5. **Summary** — a `finishedAt` timestamp is set. Markdown and JSON exports become available. The GM fills a debrief textarea (which is itself included in the export). A "New session" action clears state after a confirm-first prompt.

The navigation bar at the top of every stage exposes all five stages as clickable steps; stages beyond the current data readiness are disabled. This lets the GM revisit the character sheet mid-scenario (for instance, if the participant wants to change a trait as they get into character) without losing scenario progress.

4.9 Testing the prototype

Correctness of the prototype was verified through three layers.

The first layer is an internal smoke test using Playwright. The test walks the full v2 flow — from a fresh `localStorage` through participant-code entry, character creation, pack and scenario selection, to the run stage — and asserts that the page produces zero JavaScript console errors and zero page errors, that expected DOM landmarks (`#recs-root` , `input.persona-name` , `button.scenario-card` , `.scenario-head` , `.beat` , `.gm-panel`) render at the right stages, and that the correct number of packs (2) and scenarios (3 within the Poco pack) are exposed. The test is kept as `_smoketest_v2.py` alongside the toolkit source so it can be re-run after any future change.

The second layer is an internal dry run planned for Week 2, in which the two thesis authors play participant and GM for each other across the full flow of each Poco scenario. This probes the mechanic at a level the automated test cannot reach: whether the pacing is right, whether the narration overrides sound like the

product, whether the UXP spend moments feel meaningful, and whether the data artefact at the end reads as analysable.

The third layer is a pilot test planned for Week 4 with one or two Interaction Design peers who are not the target population but who can surface mechanical problems — unclear rules during Session Zero, ambiguous instructions on challenge beats, awkward phrasing in the complication library, missing data in the export. The pilot will inform a v2.1 revision before the main study.

4.10 What the prototype does and does not do

The prototype operationalises RECS as a usable research instrument. It covers the persona sheet, the dice mechanic, the complication library, session capture, and export. It is product-agnostic by construction and has been demonstrated on two packs. It runs with no toolchain and is testable end-to-end in a headless browser.

The prototype does not automate analysis. The Markdown and JSON exports are the starting point of thematic analysis, not its product. Coding, theme extraction, and inter-rater reliability checks remain manual, and will be discussed in Chapter 6 (Analysis). The prototype also does not capture audio or screen recording directly — these remain the researcher's responsibility using standard tools, with filenames linked by participant code.

Finally, the prototype does not include the comparison condition against which it is evaluated. The within-subjects study design pairs a RECS session with a traditional think-aloud usability test on the same product; the traditional condition uses standard screen recording and an observation template (included as Appendix C). Keeping the comparison out of the toolkit keeps the toolkit itself clean — a researcher adopting RECS for their own work is not obliged to also adopt a comparison protocol they have no interest in running.

4.11 Summary

This chapter documented the design of the RECS toolkit from the Sturdee et al. proposal through a v1 that misread the dice mechanic as success adjudication to a v2 that restores the paper's Blades-in-the-Dark lineage of perturbation injection. The resulting instrument is a zero-build-step, browser-runnable, pack-extensible toolkit that produces a structured session record from a TTRPG-style UX research session. The rest of the thesis evaluates that instrument: Chapter 5 reports the user study; Chapter 6 analyses the resulting data, comparing what RECS surfaces to what a traditional usability test surfaces on the same product with the same participants.

Word count (approx.): 3,150.